

Open in app ↗

 Medium

Logistic Regression

10 min read · Oct 7, 2024



Rishabh Singh

Follow



Listen



Share

... More

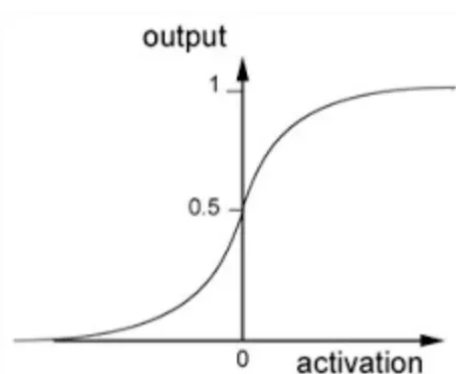
Logistic Regression is one of the most fundamental algorithms in Machine Learning and is primarily used for classification problems. Unlike Linear Regression, which predicts continuous values, Logistic Regression is used to predict a binary outcome (like yes/no, 0/1, true/false).

Understanding Logistic Regression

Logistic Regression predicts the probability that a given input belongs to a certain class. For example, it could predict whether an email is spam or not. The output is between 0 and 1, representing probabilities. To get this probability, we use a special function called the **sigmoid function**.

The Sigmoid Function

The sigmoid function maps any real-valued number into a range between 0 and 1, which is ideal for probability prediction.



$$S(t) = \frac{1}{1 + e^{-t}}$$

Sigmoid Function

For Logistic Regression, the input to the sigmoid function is a linear combination of the input features $x_1, x_2, \dots$ and their respective weights:

$$z = m_1 \cdot x_1 + m_2 \cdot x_2 + m_3 \cdot x_3 + \dots + m_n \cdot x_n$$

Then, the probability that the output belongs to the “positive” class is:

$$h(z) = \frac{1}{1 + e^{-z}}$$

Here, $h(z)$ gives the probability that the outcome is 1, while $1-h(z)$ gives the probability that the outcome is 0.

Example : Let's say we are trying to predict whether a customer will buy a product based on two features: the number of times they visited the website (x_1) and their income (x_2). We will assign weights to these features and apply the sigmoid function to predict the probability of a purchase.

Cost Function

A **cost function** (also known as a loss function) measures the error between the actual values (ground truth) and the predicted values by a model. The goal of most machine learning algorithms is to minimize this cost, which leads to better predictions.

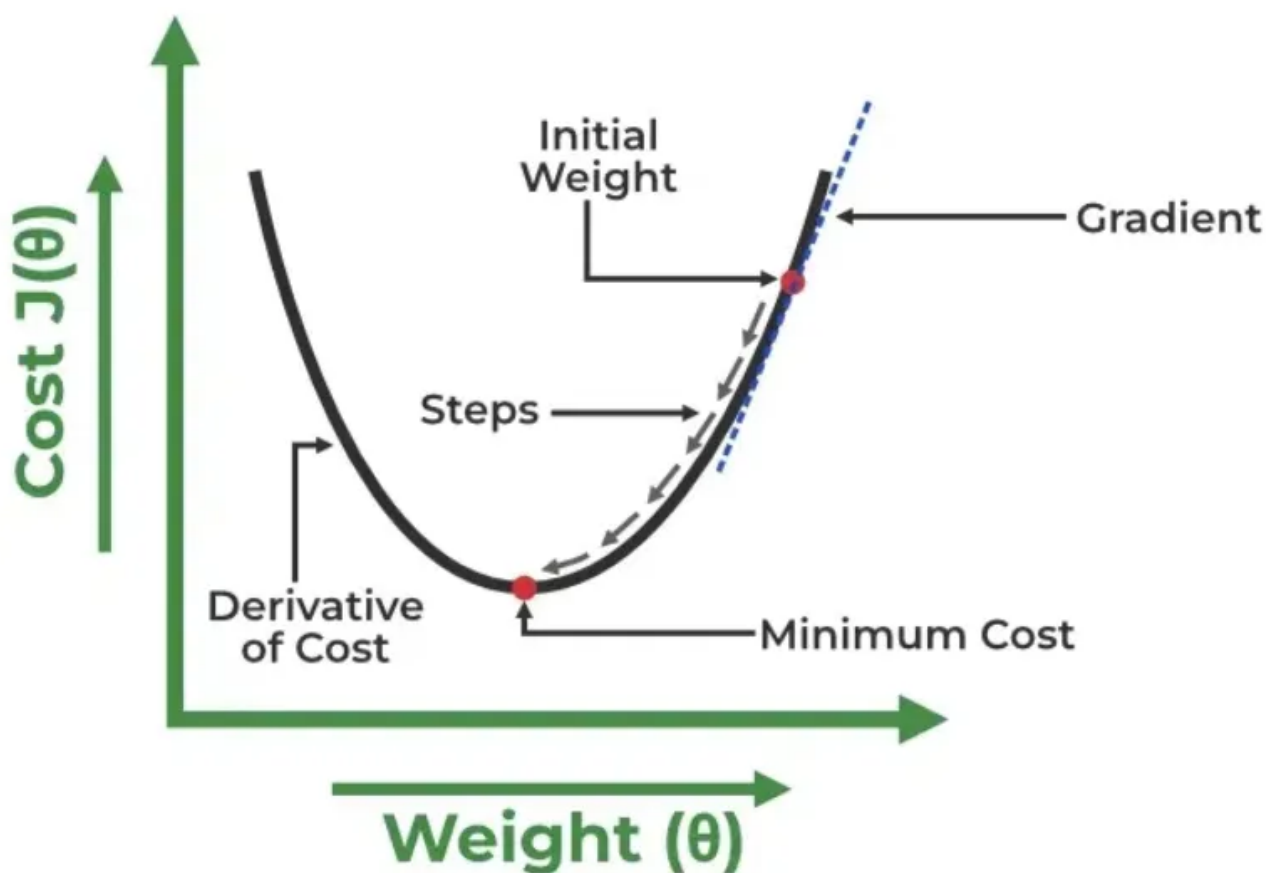
Linear Regression : In **Linear Regression**, the cost function typically used is the **Mean Squared Error (MSE)**. It calculates the average squared difference between the predicted values and the actual values, which helps adjust the model's parameters to minimize the error.

The formula for MSE is:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \tilde{y}_i)^2$$

Where:

- n is the number of training examples.
- $y(i)$ is the predicted value for the i -th training example.
- $y(i)^\wedge$ is the actual value for the i -th training example.



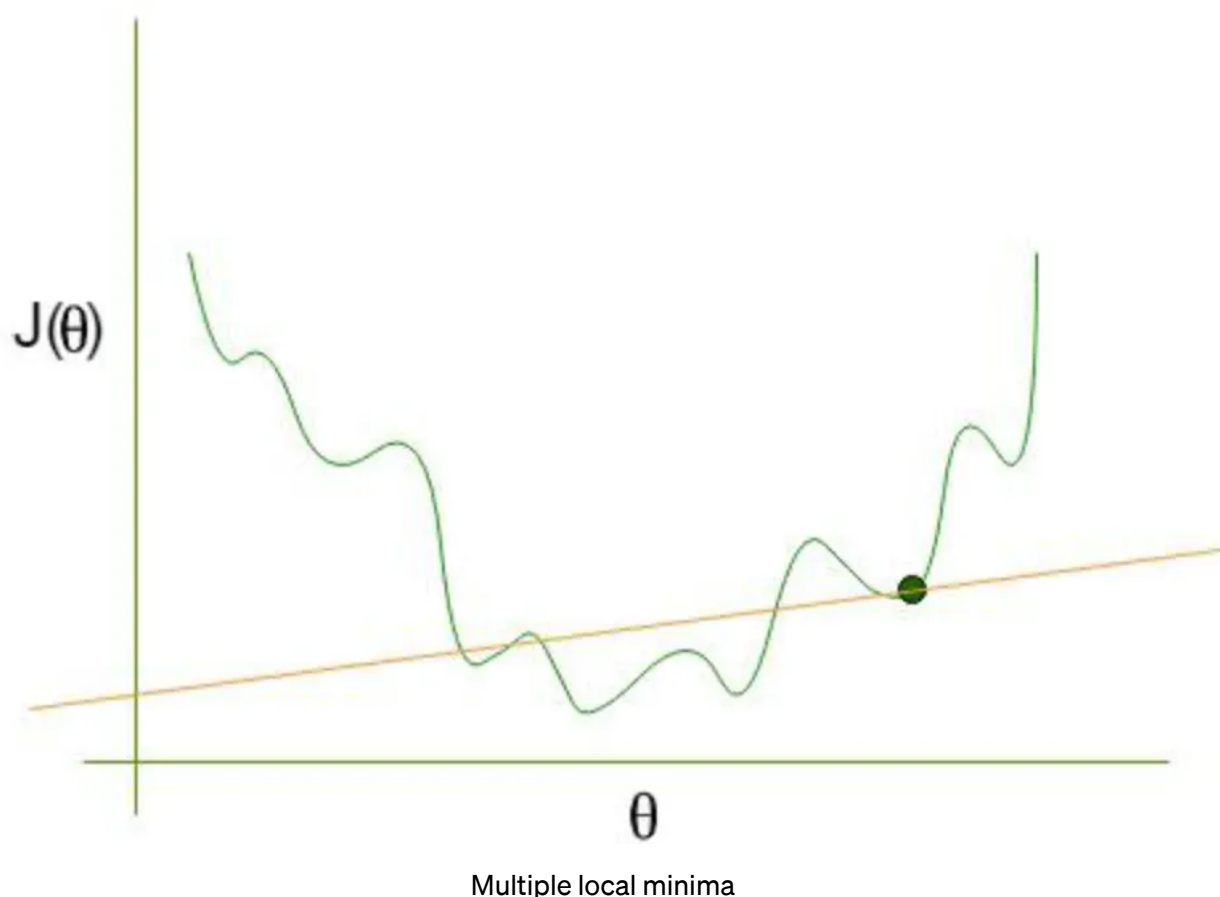
Since the graph of this cost function is convex (U-shaped), we can use **Gradient Descent** to find the optimal model parameters. Gradient Descent helps find the global minimum of the function, ensuring we have the best model.

Why MSE is Not Suitable for Logistic Regression

Unlike Linear Regression, **Logistic Regression** is used for **classification tasks**, where the output is a probability between 0 and 1. Logistic Regression uses the **sigmoid function** to predict these probabilities.

If we try to use the **Mean Squared Error** for Logistic Regression, we would face several problems:

1. **Nonlinearity of the Sigmoid Function:** The sigmoid function introduces nonlinearity, and when we plug it into the MSE formula, the cost function becomes non-convex. This means it could have multiple local minima, making it harder for Gradient Descent to find the optimal solution.



2. **Squaring Errors:** The MSE squares the difference between the predicted probability and the actual class label (0 or 1). When the prediction is far from the actual value, the error gets magnified. However, because the outputs of Logistic Regression are probabilities (values between 0 and 1), squaring these small differences can make it difficult for the model to learn effectively.

The Solution: Log Loss (Cross-Entropy)

Instead of MSE, **Logistic Regression** uses a different cost function called **Log Loss** (or Cross-Entropy). Log Loss penalizes incorrect predictions more effectively than MSE and helps the model learn to improve. It works by taking the **logarithm of predicted probabilities**, allowing the model to focus on making confident predictions closer to the true class labels.

The Log Loss function is:

$$J = - \sum_{i=1}^m y_i \log(h_{\theta}(x_i)) + (1 - y_i) \log(1 - h_{\theta}(x_i))$$

Where:

- m is the number of training examples
- y_i is the true class label for the i -th example (either 0 or 1).
- $h_{\theta}(x_i)$ is the predicted probability for the i -th example, as calculated by the logistic regression model.
- θ is the model parameters

This cost function is convex, meaning it has a single global minimum. Gradient Descent can easily optimize this function, helping the model find the best parameters for classification.

Multiclass Classification

In **multi-class classification**, the goal is to categorize data into more than two categories (e.g., classify animals into cat, dog, or bird). Most machine learning algorithms, however, are designed for **binary classification**, where the goal is to classify data into one of two categories (like “spam” vs “not spam”).

But don't worry! We can adapt these binary classification models for multi-class classification using two common techniques: *One-Vs-Rest (OvR)* and *One-Vs-One (OvO)*. Let's break down these methods in a simple way so we can understand

how they work and when to use each one.

One-Vs-Rest

OvR is a technique where we take a multi-class classification problem and split it into **multiple binary classification problems**. For each class, we create a binary classifier that tries to distinguish one class from the rest.

Let's say we have three classes: "Red," "Blue," and "Green." With OvR, we will create 3 separate binary classification problems:

- **Problem 1:** Is it "Red" or not Red (i.e., Blue or Green)?
- **Problem 2:** Is it "Blue" or not Blue (i.e., Red or Green)?
- **Problem 3:** Is it "Green" or not Green (i.e., Red or Blue)?

For each binary problem, a model will be trained. When we want to make a prediction for a new data point, each model will give a probability for its specific class (e.g., how confident it is that the data belongs to the "Red" class). We then choose the class with the highest probability (i.e., the model that is the most confident).

Example:

Imagine we are trying to classify a fruit as either an apple, orange, or banana based on features like color, size, and shape, using One-Vs-Rest:

- **Model 1** will classify whether the fruit is an apple or not (could be an orange or banana).
- **Model 2** will classify whether the fruit is an orange or not (could be an apple or banana).
- **Model 3** will classify whether the fruit is a banana or not (could be an apple or orange).

During prediction, if the "apple" model predicts with 0.7 probability, the "orange" model with 0.2, and the "banana" model with 0.1, the classifier will predict the fruit is an apple, as 0.7 is the highest probability.

Advantages:

- Simple to implement.
- Works well with many binary classifiers, like Logistic Regression or Support Vector Machines (SVM).

Disadvantages:

- We need one model per class. So, if we have 100 classes, we will need to train 100 models.
- Some models may be slower when applied to large datasets because we need to fit one model per class.

One-Vs-One (OvO)

In OvO instead of creating one binary classifier per class, we create a classifier for **each pair of classes**. This means we break the problem into binary classification tasks for every pair of classes.

Let's take the same example of three classes: "Red," "Blue," and "Green." Instead of three binary classification problems (as in OvR), OvO would create the following binary classification problems:

- **Problem 1:** "Red" vs "Blue"
- **Problem 2:** "Red" vs "Green"
- **Problem 3:** "Blue" vs "Green"

For each pair of classes, we train a separate binary classifier. Then, during prediction, each model makes its prediction, and the class that gets the most "wins" across all the models is selected as the final prediction. This is like a voting system, where each binary classifier votes for its preferred class, and the class with the most votes wins.

Example:

For the fruit classification problem (apple, orange, banana), OvO would create classifiers for:

- Apple vs Orange
- Apple vs Banana
- Orange vs Banana

When predicting the class for a new fruit:

- The "Apple vs Orange" model votes for either apple or orange.
- The "Apple vs Banana" model votes for either apple or banana.
- The "Orange vs Banana" model votes for either orange or banana.

The final classification is based on which fruit gets the most votes.

Advantages:

- Each binary classifier deals with only two classes, which makes the classifiers

simpler and sometimes more accurate.

- Can handle large datasets well, especially when binary classifiers are fast to train.

Disadvantages:

- The number of models grows quickly as the number of classes increases. For (n) classes, the number of binary classifiers is given by the formula:

$$\text{Number of Models} = n(n-1)/2$$

- More models means more storage and longer training time.

Creating Extra Features to Improve Models

When building machine learning models, it's common to create **new features** from the original ones to improve the model's performance. Let's say we have 3 features (or variables) in our data: $f1$, $f2$, and $f3$. These could represent anything like the height, weight, or age of a person.

Example: First-Degree Features

If we keep things simple, we could create an equation where these features are only multiplied by some numbers (called coefficients), like this:

$$a \cdot f1 + b \cdot f2 + c \cdot f3 + d$$

In this equation, **a**, **b**, **c**, and **d** are just numbers that the model learns during training. This equation is called **first-degree** because the features are used as they are — there are no squares or multiplied terms.

Adding More Features: Second-Degree Terms

Now, to improve the model, we can add **second-degree features**. These are new features created by multiplying the original ones with themselves or with each other. For example:

- $f1 \cdot f1$
- $f2 \cdot f2$
- $f3 \cdot f3$

- $f1 \cdot f2$ (interaction between $f1$ and $f2$)
- $f1 \cdot f3$
- $f2 \cdot f3$

So, now we have **9 features** in total instead of 3. These new features help the model capture more complex patterns in the data, like curves, which a simple straight-line model can't.

We can also add even more complex features, like third-degree terms, such as:

$$f1 \cdot f2 \cdot f3$$

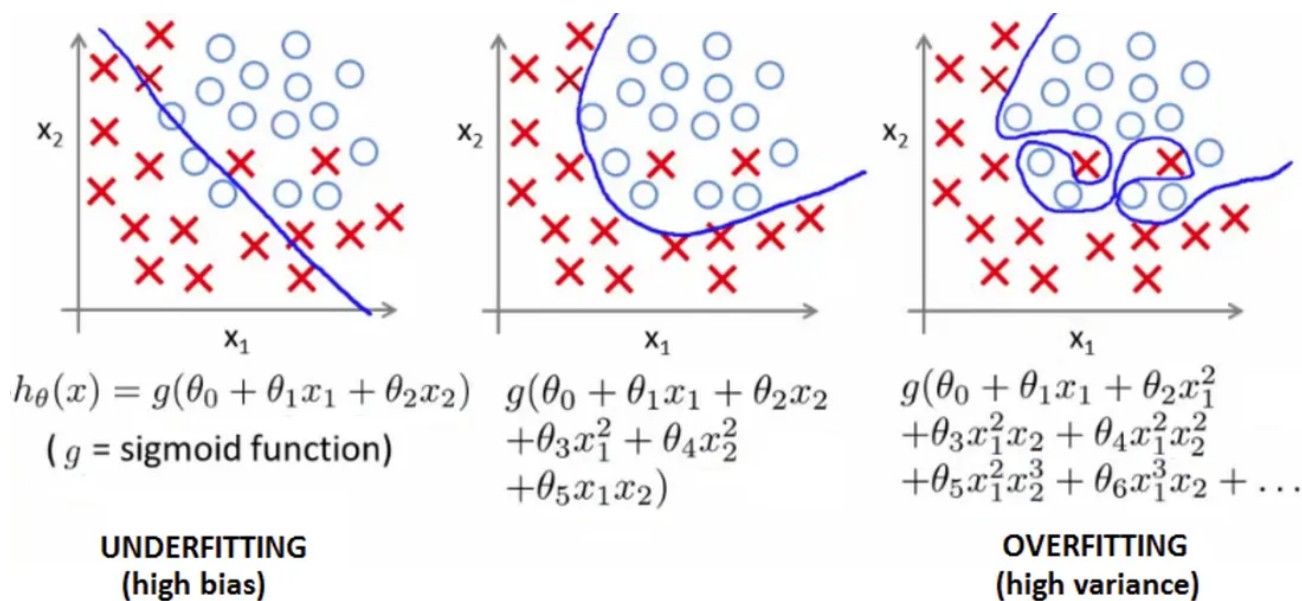
As we add more of these higher-degree terms, the model can fit more complicated shapes to the data, like curves or parabolas. This helps the model better understand the data and create more flexible decision boundaries.

The Problem of Overfitting

However, there's a **downside** to adding too many features, especially very high-degree ones. The model might start to fit the training data **too well**. It may capture not only the general patterns but also the **noise** or random quirks in the data. This is called **overfitting**.

- **Overfitting** means the model becomes so good at predicting the training data that it performs poorly when given new, unseen data (test data).
- The model is “tricked” into thinking the noise is important, so when it encounters new data, it makes mistakes because the patterns it learned aren't general enough.

Let's take a look at the example below:



1. In the first figure, we use a **degree 1 equation** (a straight line). This line is too simple and doesn't fit the data well — lots of mistakes are made.
2. In the second figure, we use a **degree 2 equation**. This curve fits the data much better and balances fitting the data without overcomplicating things. This is the **optimal solution** because it captures the overall trend even though a few points are misclassified.
3. In the third figure, we use a **degree 5 equation**. The decision boundary is very complex and tries to fit **every single point** perfectly. However, this is a case of **overfitting** because it fits the training data too well, and this complexity will likely lead to poor performance on new, unseen data.

Using Sklearn for Logistic Regression

```
# Import necessary libraries from scikit-learn
from sklearn import datasets # For loading datasets
from sklearn import model_selection # For splitting data into train and test sets
from sklearn.linear_model import LogisticRegression # For logistic regression

# Load the breast cancer dataset from sklearn's built-in datasets
cancer_ds = datasets.load_breast_cancer()

# Assign the features (input variables) to X and the target labels (output variable) to Y
X = cancer_ds.data # X contains the input features
Y = cancer_ds.target # Y contains the target labels (malignant=0, benign=1)
```

```
# Split the dataset into training and testing sets
# test_size=0.3 means 30% of the data will be used for testing, and 70% for
X_train, X_test, Y_train, Y_test = model_selection.train_test_split(X, Y, te

# Create an instance of the Logistic Regression model
# C=2 controls regularization (lower values indicate stronger regularization)
# solver='liblinear' is used because it's suitable for smaller datasets
clf = LogisticRegression(C=2, solver='liblinear')

# Train the Logistic Regression model on the training data
clf.fit(X_train, Y_train)

# Print the training accuracy (how well the model performs on training data)
print("Training Score", clf.score(X_train, Y_train))

# Print the testing accuracy (how well the model generalizes to unseen test
print("Testing Score", clf.score(X_test, Y_test))
```



Logistic Regression

Machine Learning

Cost Function

Log Loss

Overfitting



Follow



Written by Rishabh Singh

530 followers · 4 following

I'm a passionate AI/Robotics enthusiast. Dedicated to pushing the boundaries of programming, robotics/ AI contributing to innovative advancements in the field.

Responses (5)



Mehmet Pekmezci

What are your thoughts?



James kabuga he/him

May 4



Well done Rishabh , the article has broken down the concepts very well and it easy to comprehend.



1

[Reply](#)



anam tech

Nov 4, 2025



Perfect Article



[Reply](#)



Avikabtcse

May 23, 2025



Very well articulated article.



[Reply](#)

See all responses

